

Project Report: Traffic Vision Analytics

1. Cover Page

Field	Detail
Project Title	Traffic Vision Analytics -- Vehicle Detection
Course	CSE3010 - Computer Vision
Program	B.Tech Computer Science and Engineering
Student	Soham Singh Mehra
GitHub	[Soham-Singh-Mehra-AI/traffic-vision-analytic
Term	Fall 2026

2. Introduction

Fixed-position traffic cameras generate hours of continuous footage that's mostly watched by nobody. The goal of this project is to turn that footage into structured, actionable data -- vehicle counts, types, speeds, and congestion metrics -- without requiring any manual observation.

The pipeline takes a traffic video as input and runs through these stages:

1. Detection -- YOLO11n identifies vehicles (cars, buses, trucks, motorcycles) in each frame. An MOG2 background subtraction baseline is also available for comparison.
2. Tracking -- A multi-object tracker maintains persistent IDs for each vehicle across frames, handling temporary occlusions.
3. Perspective correction -- A 4-point planar homography maps pixel positions to real-world ground coordinates in metres.
4. Speed estimation -- Using the corrected coordinates, the system calculates how fast each vehicle is moving in m/s and km/h.
5. Analytics -- The results are aggregated into traffic engineering metrics: density, Level of Service (HCM standard), speed percentiles, and congestion classification.

Everything runs headless from the command line, producing an annotated video, a CSV log, and a JSON summary.

3. Problem Statement

Manual traffic monitoring doesn't scale. A person watching a camera feed can maybe track a handful of vehicles; they can't maintain frame-by-frame records, and they can't measure speeds without additional hardware.

Radar guns and loop detectors give speed data but only at fixed points -- they don't capture trajectories, vehicle types, or density patterns over a stretch of road.

This project asks: can a single surveillance camera, combined with computer vision, replace all of that? Specifically -- can we build a pipeline that ingests video, tracks every vehicle through the scene, corrects for the camera's perspective distortion, computes physical speeds, and exports the whole thing as structured data ready for analysis?

The additional constraint: it has to run headless, from a terminal, without needing a graphical display. This matters because evaluation environments (like VITYarthi) and cloud servers typically don't have monitors attached.

4. Functional Requirements

- * FR1 -- Video ingestion: Accept video files up to 4K UHD @ 50 FPS. Support optional frame skipping (--skip N) to trade accuracy for speed on slower hardware.
- * FR2 -- Vehicle detection: Detect cars, buses, trucks, and motorcycles using YOLO11n. Configurable confidence threshold and inference resolution.
- * FR3 -- MOG2 baseline: Provide a background subtraction detector using OpenCV's MOG2 with morphological filtering, so the project can compare classical and deep learning approaches.
- * FR4 -- Multi-object tracking: Assign and maintain a unique ID for each vehicle across frames. Handle occlusions gracefully (tracks survive missed detections for up to N frames).
- * FR5 -- Homography-based perspective correction: Use 4 ground-plane reference points to compute a 3x3 homography matrix that maps image pixels to real-world metre coordinates.
- * FR6 -- Speed estimation: Compute vehicle velocity from corrected positions over a sliding time window. Report in m/s and km/h (calibrated) or pixels/s (uncalibrated).
- * FR7 -- Traffic analytics: Calculate aggregate metrics -- HCM Level of Service (A-F), 85th-percentile speed, traffic density, hourly flow rate, fleet composition.
- * FR8 -- Output export: Produce annotated video (MP4), per-frame vehicle data (CSV), and a summary report (JSON).

5. Non-Functional Requirements

* NFR1 -- Headless operation: No GUI calls, no cv2.imshow() in the pipeline. Uses QT_QPA_PLATFORM=offscreen so it works on servers, Docker, and automated grading systems.

* NFR2 -- Fault tolerance: Missing calibration files don't crash the system -- it falls back to default geometry or uncalibrated mode. Missing video paths produce a clean error message, not a traceback.

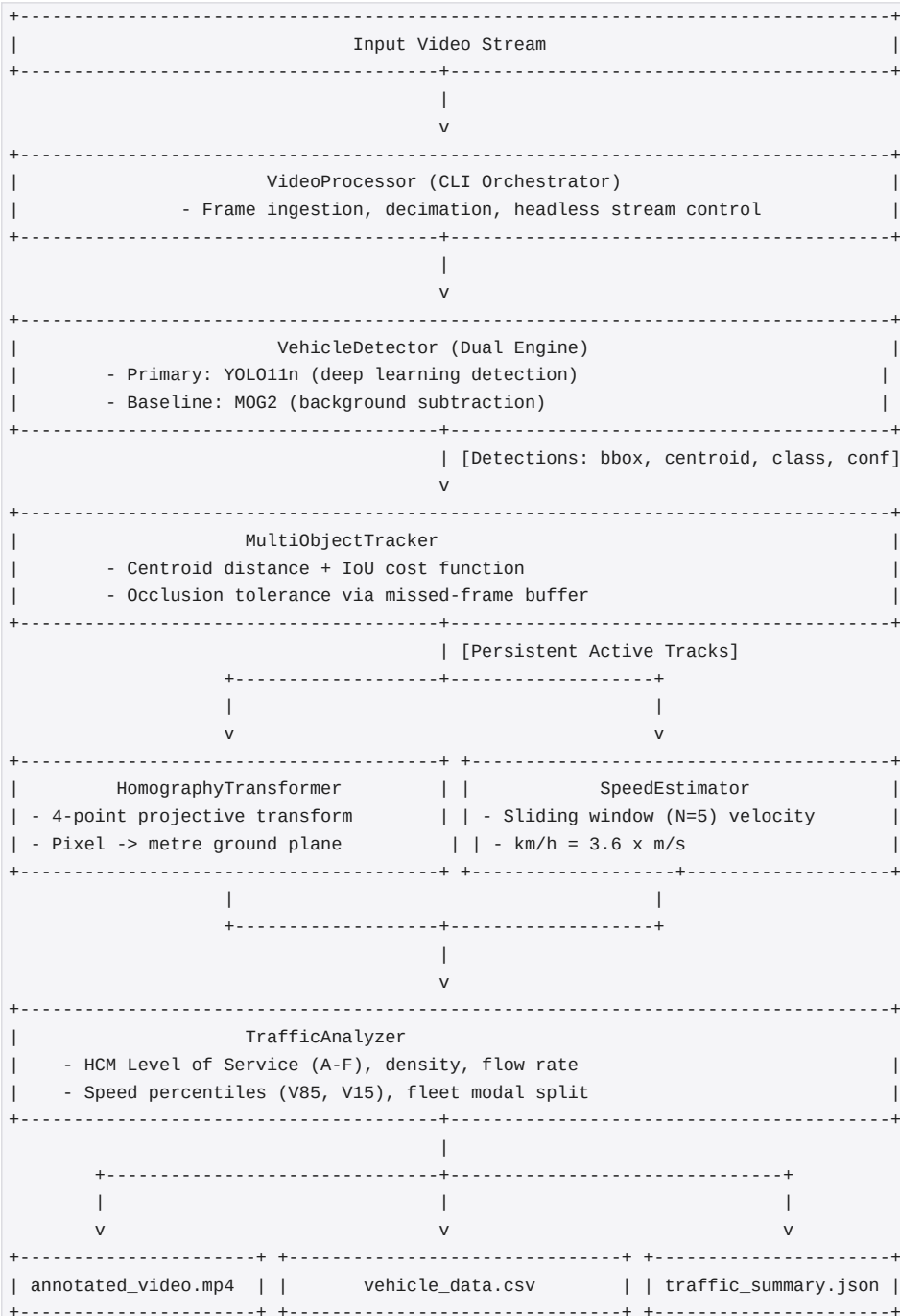
* NFR3 -- Performance: Achieves 8-15 FPS on a standard CPU. Supports frame skipping and GPU offloading (--device cuda) for faster processing.

* NFR4 -- Reproducibility: Standard Python venv + pinned requirements.txt. 13 automated tests via pytest.

* NFR5 -- Maintainability: 8 focused modules in src/, each with a single responsibility. PEP 8 style, type hints throughout.

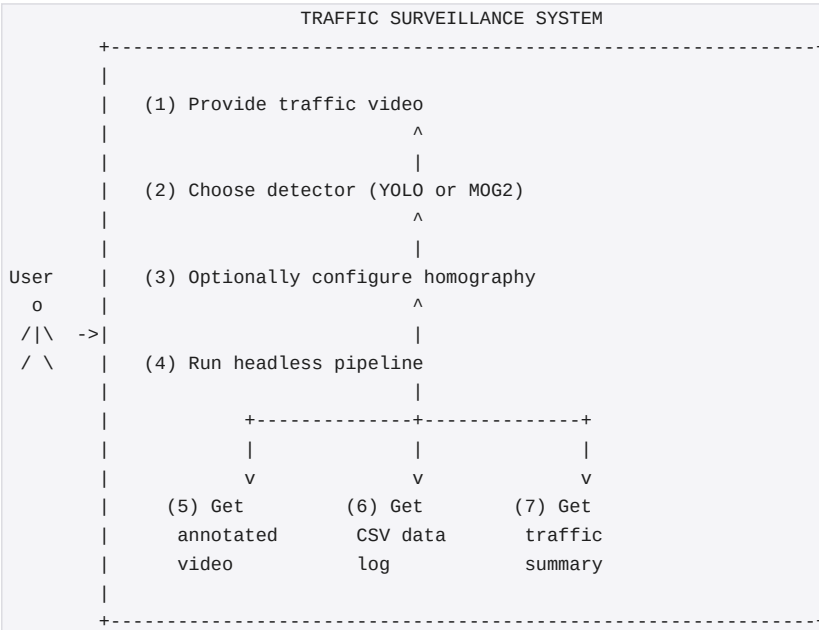
6. System Architecture

The pipeline is a linear chain of modules, each processing the output of the previous one:



7. Design Diagrams

7.1 Use Case Diagram



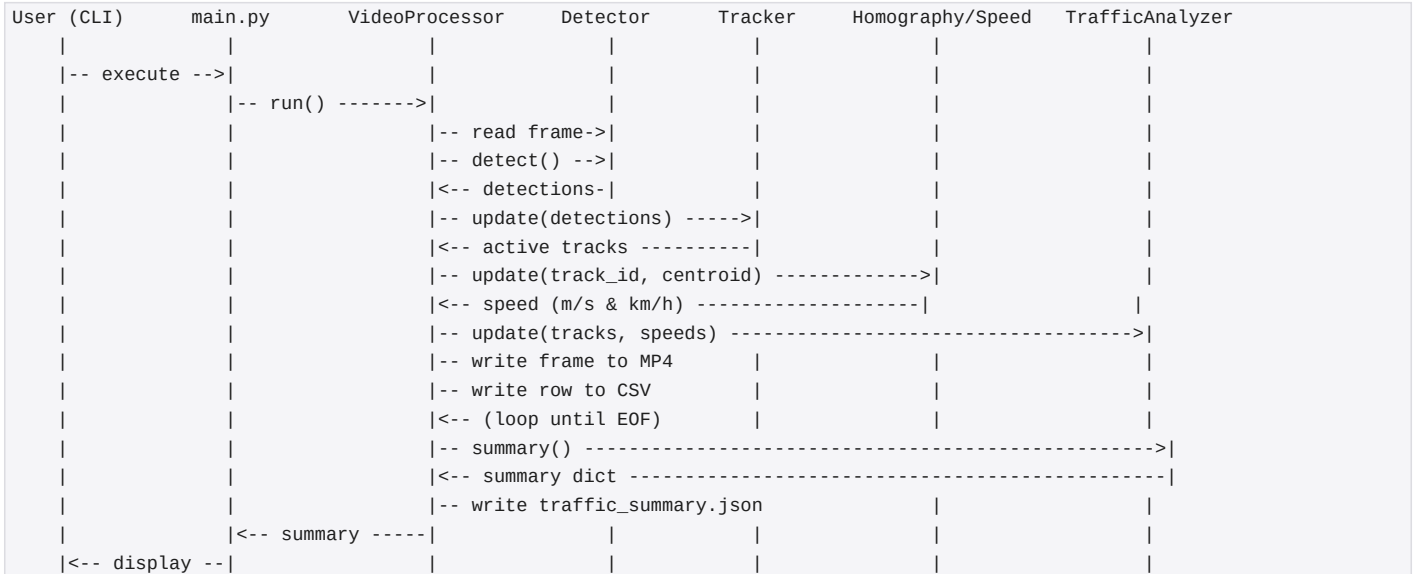
7.2 Process Flow

```

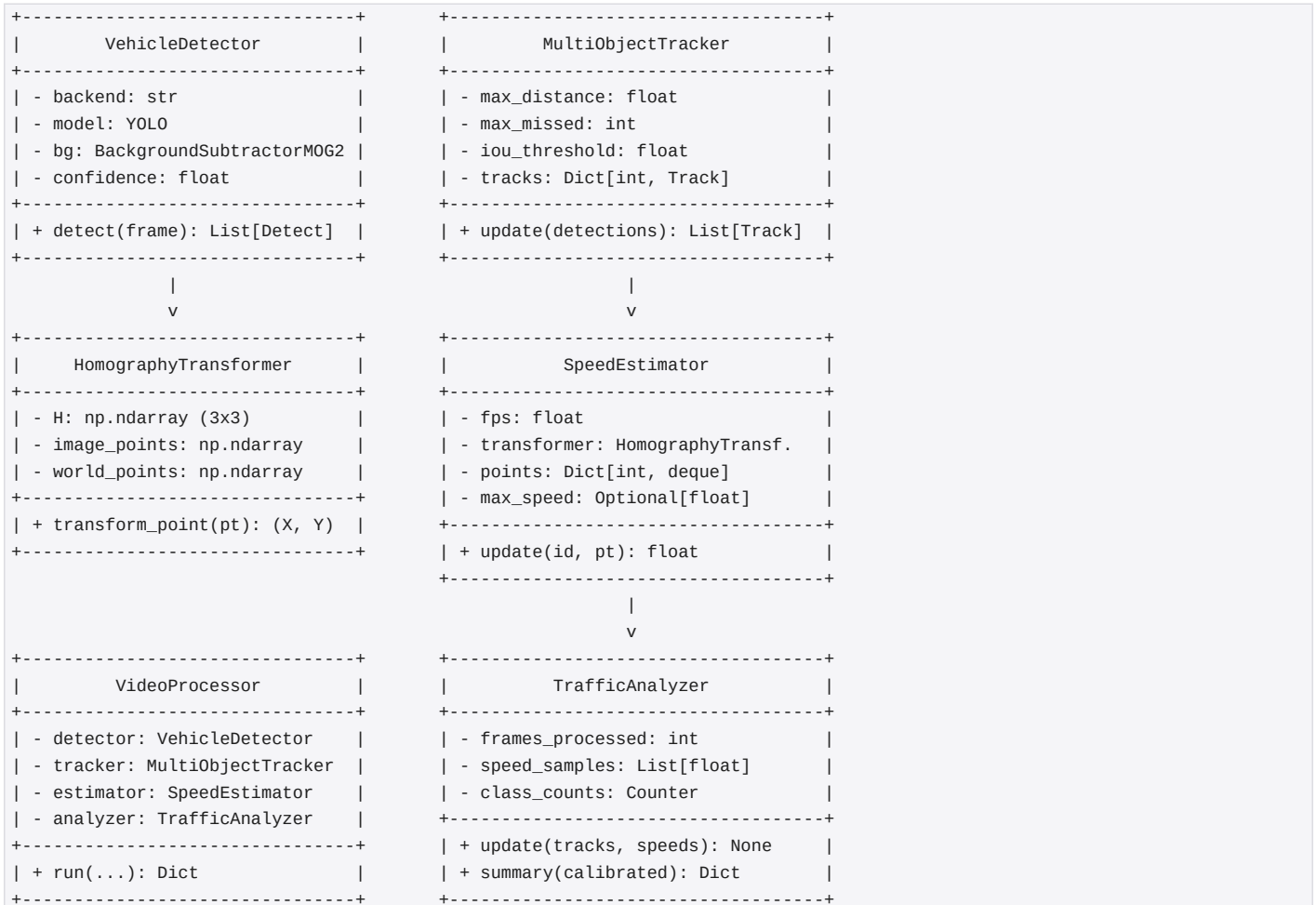
[Start]
|
v
[Validate CLI arguments & check video exists] --> (Missing? -> error exit)
|
v
[Load calibration]
|--> Try specified file
|--> Fall back to config/calibration.json
|--> Synthesize default geometry if nothing found
|
v
[Initialize detector (YOLO or MOG2)]
|
v
[Open video, read metadata (FPS, resolution, frame count)]
|
v
+--> [Read next frame] --> (End of video? -> write summary & exit)
|
|
|   v
|   [Detect vehicles -> bounding boxes, centroids, classes]
|   |
|   v
|   [Update tracker -> associate detections to existing tracks]
|   |
|   v
|   [Compute speed for each track via homography]
|   |
|   v
|   [Update traffic analyzer with frame data]
|   |
|   v
|   [Draw annotations -> write frame to output video]
|   |
|   v
|   [Write CSV row for each tracked vehicle]
|   |
+--- (loop)

```

7.3 Sequence Diagram



7.4 Class / Component Diagram



7.5 Output Schema

vehicle_data.csv:

Field	Type	Description
frame	int	Frame index
vehicle_id	int	Track ID
class	str	Vehicle type

confidence	float	Detection confidence
x, y	int	Centroid position
width, height	int	Bounding box size
speed	float	Velocity (m/s or px/s)
speed_unit	str	Unit label
speed_kmh	float/str	km/h or "N/A"

traffic_summary.json: frames processed, unique tracks, average/peak active vehicles, density, HCM LOS, congestion level, flow rate, detection counts, modal split, speed statistics, calibration status, resolution, FPS.

8. Design Decisions

1. YOLO11n over heavier models: YOLO11n gives good detection accuracy on vehicle classes while running under 15ms per frame on CPU. Larger models (YOLO11x, Mask R-CNN) don't meaningfully improve bounding-box centroid tracking, and the latency cost isn't worth it for this use case.

2. Keeping MOG2 as a baseline: Background subtraction is a core topic in the CV course (Module 4). Including MOG2 alongside YOLO makes it easy to demonstrate the limitations of motion-only detection -- for example, MOG2 loses vehicles that stop at traffic lights because they blend into the background model.

3. Homography over depth estimation: Monocular depth networks give relative depth, not calibrated distances, and they need GPU inference. A 4-point homography gives exact pixel-to-metre mapping with zero runtime cost once the matrix is computed. It requires knowing 4 reference points on the road, but that's a one-time calibration step.

4. Sliding window speed smoothing (N=5): Frame-to-frame centroid movement is noisy because bounding boxes jitter slightly between frames. Averaging displacement over 5 frames smooths out the noise while still capturing real acceleration.

5. Headless-only pipeline: The evaluation platform runs without a display server. All cv2.imshow() calls are removed from the pipeline, and QT_QPA_PLATFORM=offscreen is set at startup to prevent crashes.

9. Implementation

The code is organized into 8 modules in src/:

1. detector.py -- VehicleDetector with two backends. YOLO filters detections to vehicle COCO classes. MOG2 uses morphological operations to clean up the foreground mask.
2. tracker.py -- MultiObjectTracker using centroid distance + IoU for association. Tracks have active/missed states and survive occlusions for up to 12 frames.
3. homography.py -- HomographyTransformer wrapping cv2.getPerspectiveTransform and cv2.perspectiveTransform.
4. speed_estimator.py -- SpeedEstimator with a deque-based sliding window. Includes a max-speed filter to suppress singularities near the vanishing point.
5. traffic_analyzer.py -- TrafficAnalyzer computing HCM LOS, speed percentiles, density, flow rate, and fleet composition.
6. visualizer.py -- Resolution-adaptive rendering of bounding boxes, ID labels, trajectory trails, and a translucent HUD banner.
7. video_processor.py -- Main processing loop: reads frames, calls detector -> tracker -> speed -> analyzer -> visualizer, writes to MP4 and CSV.
8. main.py -- CLI entry point with argument parsing and fault-tolerant calibration loading.

10. Results

Measurement Summary (4K test video, 300 frames)

Metric	Calibrated	Uncalibrated
Frames processed	300	300
Unique tracked vehicles	29	29
Avg active vehicles/frame	7.62	7.62
Peak concurrent vehicles	10	10
Density (veh/lane-km)	43.87	43.87
HCM Level of Service	LOS F	--
Total detections	2,285	2,285
Cars	1,915 (83.8%)	1,915 (83.8%)
Buses	351 (15.4%)	351 (15.4%)

Trucks	19 (0.8%)	19 (0.8%)
Mean speed	9.08 m/s (32.70 km/h)	301.92 px/s
85th percentile speed	15.21 m/s (54.77 km/h)	512.4 px/s
15th percentile speed	1.14 m/s (4.10 km/h)	38.2 px/s
Speed std dev	7.59 m/s	184.2 px/s

Sample Output

11. Testing

13 automated tests run via `python -m pytest -v`:

- * IoU tests (`test_tracker.py`) -- boundary cases for intersection-over-union
- * Tracker persistence (`test_tracker.py`) -- verifying IDs stay consistent across frames
- * Detection schema (`test_tracker.py`) -- dataclass field validation
- * Homography (`test_homography.py`) -- checking that the transform maps points correctly
- * Pixel speed (`test_speed.py`) -- basic displacement-over-time check
- * Calibrated speed (`test_speed.py`) -- verifying m/s -> km/h conversion
- * Traffic analyzer (`test_analyzer.py`) -- congestion level, class counts, speed aggregation
- * HCM LOS (`test_analyzer.py`) -- Level of Service thresholds
- * Speed percentiles (`test_analyzer.py`) -- V85 and V15 calculations
- * MOG2 init (`test_pipeline_integration.py`) -- background subtractor setup
- * Config fallback (`test_pipeline_integration.py`) -- graceful handling of missing calibration

All 13 pass (0.11s).

12. Challenges & Solutions

1. Vanishing point speed spikes -- Vehicles far from the camera map to extreme world coordinates via homography, producing artificially high speeds. Fixed by adding an upper-bound speed filter (38 m/s 137 km/h) and requiring a minimum track history before computing velocity.
2. Headless crashes -- OpenCV's Qt backend tries to connect to a display server. Crashes on Docker/SSH/CI. Fixed by setting `QT_QPA_PLATFORM=offscreen` before importing `cv2`, and removing all `imshow()` calls from the pipeline.
3. Bounding box jitter -- Small frame-to-frame variations in detection boxes cause noisy speed readings. Fixed with a 5-frame sliding window that averages centroid displacement.
4. Missing config files -- The calibration loader now tries 3 fallback strategies (explicit path -> default path -> auto-generate from video dimensions), so it never crashes due to a missing file.

13. Key Takeaways

- * Planar homography is a practical way to get real-world measurements from a single camera, as long as you have reference points on a flat surface. The math is straightforward but the results degrade badly near the horizon.
- * MOG2 works for detecting moving objects but fails for stationary vehicles. YOLO detects based on appearance, so it works regardless of motion -- this is the core trade-off between classical and learned approaches.
- * Transportation engineering has well-established metrics (HCM LOS, V85) that turn raw vehicle data into actionable assessments. Integrating these gave the project outputs that mean something beyond just "detected N vehicles."
- * Building for headless execution from the start avoided a whole class of deployment bugs.

14. Future Work

1. Kalman filter + DeepSORT -- Better tracking through long occlusions using motion prediction and appearance embeddings.
2. Multi-camera tracking -- Handing off vehicle IDs across camera views.
3. Automatic calibration -- Detecting lane markings via Hough transforms to compute homography without manual landmark selection.
4. Edge deployment -- Exporting YOLO11n to TensorRT/ONNX for real-time inference on NVIDIA Jetson hardware.

15. References

1. Szeliski, R. (2022). **Computer Vision: Algorithms and Applications** (2nd ed.). Springer.
2. Gonzalez, R. C., & Woods, R. E. (2018). **Digital Image Processing** (4th ed.). Pearson.

3. Transportation Research Board. (2016). *Highway Capacity Manual* (6th ed.). National Academies.
4. Redmon, J., et al. (2016). *You Only Look Once: Unified, Real-Time Object Detection*. CVPR.
5. Ultralytics. (2024). *YOLO11 Documentation*. <https://docs.ultralytics.com>
6. Bradski, G. (2000). *The OpenCV Library*. Dr. Dobb's Journal.